Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №4

«Розфарбовування графа, алгоритми розфарбування».

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 18
Перевірив:
Пономаренко А. М.

Київ – 2025

Лабораторна робота №4

Тема: «Розфарбовування графа, алгоритми розфарбування».

Мета: вивчення способів правильного розфарбовування графа

Загальне завдання:

1. створити програму для правильного розфарбовування графа на основі одного з алгоритмів розфарбування.

Теоретичні основи:

Різноманітні завдання, що виникають при плануванні виробництва, складанні графіків огляду, зберіганні та транспортуванні товарів та ін., часто можуть бути представлені як задачі теорії графів, тісно пов'язані з так званим «завданням розфарбовування». Графи, що розглядаються в даній лабораторній роботі, є неорієнтованими і такими, що не мають петель.

Граф G називають *r-хроматичним*, якщо його вершини *можуть бути розфарбовані* з використанням r кольорів (фарб) так, що не знайдеться двох суміжних вершин одного кольору. Найменше число r, таке, що граф G є *r-хроматичним*, називають *хроматичним числом* графа G і позначають $\gamma(G)$.

Завдання знаходження хроматичного числа графа називають *задачею про розфарбовування* (або *завданням розфарбовування*) графа. Відповідне цьому числу розфарбування вершин розбиває множину вершин графа на r підмножин, кожна з яких містить вершини одного кольору. Ці множини є незалежними, оскільки в межах однієї множини немає двох суміжних вершин.

Завдання знаходження хроматичного числа довільного графа стало предметом багатьох досліджень в кінці XIX і в XX столітті. З цього питання отримано багато цікавих результатів.

Хроматичне число графа не можна знайти, знаючи тільки кількість вершин і ребер графа. Недостатньо також знати степінь кожної вершини, щоб обчислити хроматичне число графа. При відомих величинах n (кількість вершин), m (кількість ребер) і $\deg(x_1)$, ..., $\deg(x_n)$ (степені вершин графа) можна отримати тільки верхню і нижню оцінки для хроматичного числа графа.

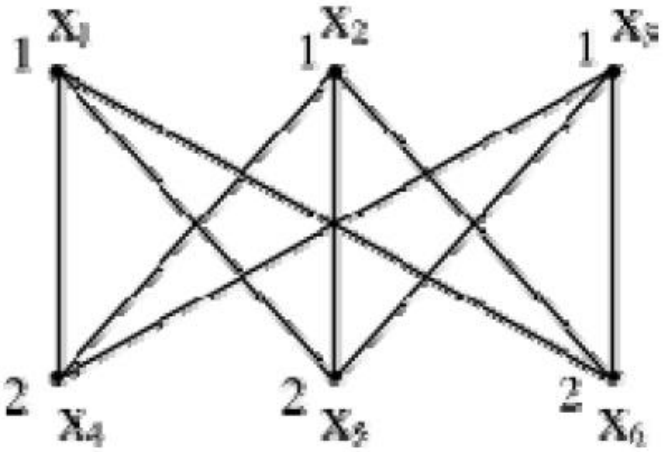


Рис. 3.1 – Дводольний біхроматичний граф Кеніга

Приклад розфарбовування графа наведений на рисунку 3.1. Цей граф є однією із заборонених фігур, що використовуються для визначення планарності. Цифрами «1» і «2» позначені кольори вершин.

Максимальна кількість незалежних вершин графа α(G), що дорівнює потужності найбільшої множини попарно несуміжних вершин, збігається також з потужністю найбільшої множини вершин в G , які можуть бути пофарбовані в один колір, отже:

$$\gamma(G) \geq \left\lceil \frac{n}{\alpha(G)} \right\rceil \quad (4.1)$$

де n - кількість вершин графа G , а $\lceil x \rceil$ позначає найбільше ціле число, яке не більше за x .

Ще одна нижня оцінка для γ(G) може бути отримана наступним чином:

$$\gamma(G) \geq \frac{n^2}{n^2 - 2m} \quad (4.2)$$

Верхня оцінка хроматичного числа може бути обчислена за формулою:

$$\gamma(G) \leq 1 + \max\left[d(x_j) + 1\right] \quad (4.3)$$

Застосування оцінок для хроматичного числа значно звужує межі рішення. Для визначення оцінки хроматичного числа також можуть використовуватися інші топологічні характеристики графа, наприклад, властивість планарності.

Граф, який можна зобразити на площині так, що жодні два його ребра не перетинаються між собою, називають *планарним.*

Теорема про п'ять фарб. Кожен планарний граф можна розфарбувати за допомогою п'яти кольорів так, що будь-які дві суміжні вершини будуть пофарбовані в різні кольори, тобто якщо граф G - планарний, то $\gamma(G) \leq 5$.

Гіпотеза про чотири фарби (недоведена). Кожен планарний граф можна

розфарбувати за допомогою чотирьох кольорів так, що будь-які дві суміжні вершини будуть пофарбовані в різні кольори, тобто якщо граф G – планарний, то $\gamma(G) \leq 4$.

У 1852 р. про гіпотезу чотирьох фарб говорилося в листуванні Огюста де Моргана з сером Вільямом Гамільтоном. З того часу ця «теорема» стала, поряд з теоремою Ферма, однією з найзнаменитіших невирішених задач в математиці.

Повний граф K_n завжди розфарбовується в n кольорів, тобто кількість кольорів дорівнює кількості його вершин.

Алгоритм прямого неявного перебору є найпростішим алгоритмом вершинного розфарбування графів. Цей алгоритм дозволяє реалізувати правильне розфарбування графа з вибором мінімальної в рамках даного алгоритму кількості фарб.

Розглянемо граф $G(V, E)$, який показаний на рис. 3.2.

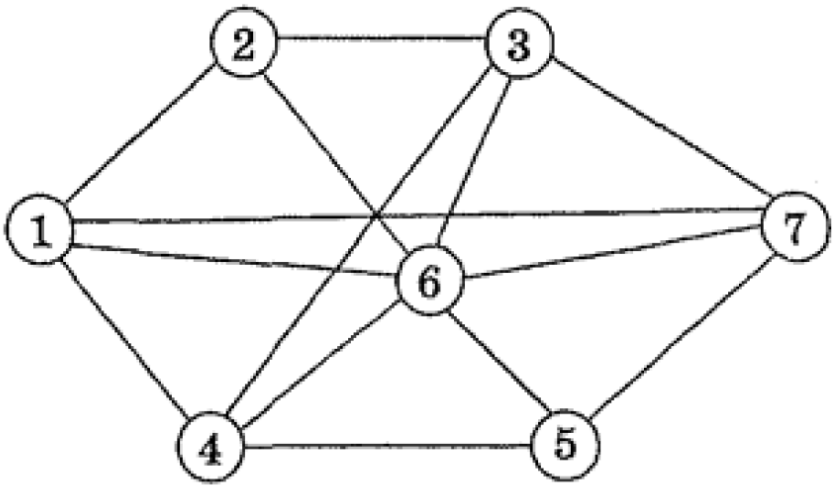


Рисунок 3.2 – граф

Множину вершин графа V = {1,2,3,4,5,6,7} потрібно розфарбувати з використанням алгоритму послідовного розфарбування.

Сформуємо матрицю суміжності A:

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 \end{pmatrix}.$$

Крок 1. Для вершини 1, відповідно до представленого вище алгоритму, множина розфарбованих суміжних вершин завжди є пустою. Тому функція Color(1) завжди повертатиме фарбу 1. Встановимо, що 1 кодує фарбу червоного кольору.

Крок 2. Розглянемо вершину 2. Для цієї вершини єдиною меншою за номером суміжною вершиною є вершина 1. Ця вершина розфарбована червоним кольором. Тому множина W містить єдиний елемент 1. Тому функція Color(2) повертає наступну за номером фарбу 2 синього кольору.

Крок 3. Вершина 3 має єдину суміжну вершину з меншим номером. Це вершина 2. Множина W містить єдиний елемент 2. Тому функція Color(3) повертає фарбу з номером 1 червоного кольору.

Крок 4. Вершина 4 має дві суміжні вершини з меншими номерами: 1 і 3. Оскільки обидві вершини розфарбовані в колір 1, то множина W містить єдиний елемент 1. Тому функція Color(4) повертає наступну за номером фарбу 2 синього кольору.

Крок 5. Вершина 5 має єдину суміжну вершину з меншим номером. Це вершина 4. Множина W містить єдиний елемент 2. Тому функція Color(5) повертає фарбу з номером 1 червоного кольору.

Крок 6. Вершина 6 має такі суміжні вершини з меншими номерами:1,3,4 і 5. Ці вершини розфарбовані в колір 1 та колір 2. Отже, множина W містить два елементи: 1 і 2. Тому функція Color(6) повертає наступну за номером фарбу 3 зеленого кольору.

Крок 7. Вершина 7 має такі суміжні вершини з меншими номерами:1,3, 5 і 6. Ці вершини розфарбовані в колір 1 та колір 3. Отже, множина W містить два елементи: 1 і 2. Тому функція Color(7) повертає фарбу 2 синього кольору.

В результаті роботи даного алгоритму одержуємо правильно розфарбований граф, що показаний на рис. 3.3.

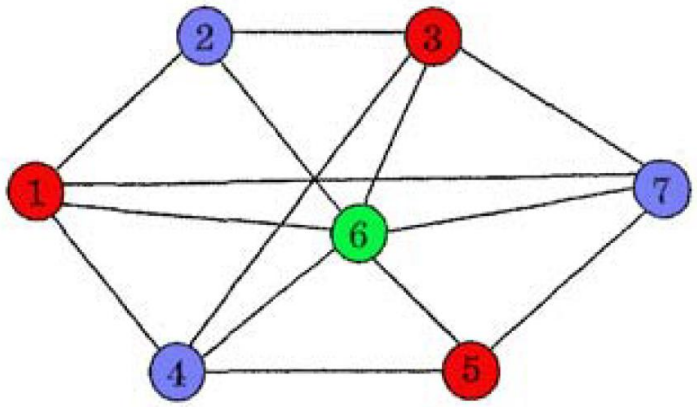


Рисунок 3.3 – результуючий граф

Загальний порядок виконання роботи:

A. Вибрати номер Z індивідуального варіанта відповідно до виразу:

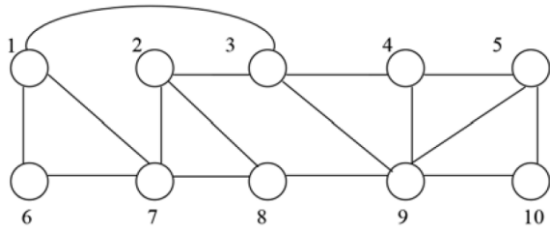
I = NZK mod6+1, де NZK – номер залікової книжки.

B. Програма обчислення варіанта:

```
G=4218
print("Номер залікової книжки:",G)
print("Мій варіант:",(G%6) +1)
```

Індивідуальне завдання:

1. Вивчити основні означення та теореми про розфарбування графів. Створити програму розфарбування графів, що реалізує алгоритм прямого перебору.

№ вар	Опис варіанту
1	А) Виконати завдання 1 до лабораторної роботи. Б) Програма повинна дозволяти розфарбування довільного графа. В) Перевірити роботу програми на даному графі *G* .  Вивести у графічному режимі розфарбований граф або включити у протокол розфарбований вручну граф за результатами роботи програми.

Розрахунок варіанту:

Код програми:

```
G=4218
print("Номер залікової книжки: ",G)
print("Мій варіант:",(G%6) +1)
```

Результат виконання програми:

```
"D:\Documents\Education\KPI\2 Ter
Номер залікової книжки: 4218
Мій варіант: 1
```

Рисунок 4.1 – результат обрахунку варіанта

Код програми:

Вміст файлу LR4.py:

```
import tkinter as tk
from tkinter import messagebox, filedialog
import networkx as nx
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import random
import varCheck as variant

class GraphApp:
    def __init__(self, master):
        self.master = master
        self.master.title("ЛР4")
        main = tk.Frame(master)
        main.pack(fill="both", expand=True)
        canv = tk.Frame(main)
        canv.pack(side="left", padx=10, pady=10, anchor="n")
        tk.Label(canv, text=(f"Куліков Максим Миколайович\nГрупа: ІО-{variant.G}\nНомер у
списку: {variant.N}\nВаріант: {variant.get_variant_number()}")).pack(anchor="w", pady=(0,10))
        tk.Label(canv, text="Кількість вершин:").pack(anchor="w")
        self.entry_n = tk.Entry(canv)
        self.entry_n.pack()
        tk.Button(canv, text="Генерувати випадкову матрицю",
command=self.generate_random_matrix).pack(fill="x", pady=(5,5))
        tk.Label(canv, text="Матриця суміжності:").pack(anchor="w")
        self.text_matrix = tk.Text(canv, width=20, height=10)
        self.text_matrix.pack()
        tk.Button(canv, text="Побудувати граф на основі матриці",
command=self.build_from_matrix).pack(fill="x", pady=(5,5))
        tk.Button(canv, text="Завантажити матрицю з файлу",
command=self.load_matrix).pack(fill="x", pady=(5,5))
        tk.Button(canv, text="Зберегти матрицю у файл", command=self.save_matrix).pack(fill="x",
pady=(5,5))
        tk.Button(canv, text="Розфарбувати граф", command=self.color_graph).pack(fill="x",
pady=(5,5))
        graph_frame = tk.Frame(main)
        graph_frame.pack(side="right", padx=10, pady=10, expand=True, fill="both")
        self.graph = nx.Graph()
        self.fig, self.ax = plt.subplots(figsize=(6,6))
        self.canvas = FigureCanvasTkAgg(self.fig, master=graph_frame)
        self.canvas.get_tk_widget().pack(fill="both", expand=True)

    def generate_random_matrix(self):
        try:
            n = int(self.entry_n.get())
            if n<=0: raise ValueError
        except:
            messagebox.showerror("Помилка","Невірне число вершин")
            return
        mat = [[0]*n for _ in range(n)]
        for i in range(n):
            for j in range(i+1,n):
                if random.choice([0,1]):
                    mat[i][j]=mat[j][i]=1
        text = "\n".join(" ".join(str(x) for x in row) for row in mat)
```

```
        self.text_matrix.delete("1.0","end")
        self.text_matrix.insert("1.0", text)
        self.build_from_matrix()

    def parse_matrix(self):
        text = self.text_matrix.get("1.0","end").strip().splitlines()
        mat = []
        for line in text:
            row = [int(x) for x in line.split()]
            mat.append(row)
        return mat

    def build_from_matrix(self):
        try:
            mat = self.parse_matrix()
            n = len(mat)
            for row in mat:
                if len(row)!=n: raise ValueError
        except:
            messagebox.showerror("Помилка","Невірна матриця")
            return
        self.graph.clear()
        nodes = [str(i+1) for i in range(n)]
        self.graph.add_nodes_from(nodes)
        for i in range(n):
            for j in range(i+1,n):
                if mat[i][j]==1:
                    self.graph.add_edge(nodes[i],nodes[j])
        self.draw_graph()

    def load_matrix(self):
        path = filedialog.askopenfilename(filetypes=[("Текст","*.txt")])
        if not path: return
        try:
            with open(path, encoding="utf-8") as f:
                data = f.read()
            self.text_matrix.delete("1.0","end")
            self.text_matrix.insert("1.0", data)
            self.build_from_matrix()
        except:
            messagebox.showerror("Помилка","Не вдалося завантажити файл")

    def save_matrix(self):
        try:
            mat = self.parse_matrix()
        except:
            messagebox.showerror("Помилка","Немає матриці")
            return
        path = filedialog.asksaveasfilename(defaultextension=".txt",filetypes=[("Текст","*.txt")])
        if not path: return
        try:
            with open(path,"w",encoding="utf-8") as f:
                for row in mat:
                    f.write(" ".join(str(x) for x in row)+"\n")
            messagebox.showinfo("Готово","Матрицю збережено")
        except:
            messagebox.showerror("Помилка","Не вдалося зберегти файл")

    def generate_assignments(self, n, k):
```

```
        assignment = [0] * n
        while True:
            yield assignment[:]
            i = n - 1
            while i >= 0 and assignment[i] == k - 1:
                assignment[i] = 0
                i -= 1
            if i < 0:
                break
            assignment[i] += 1

    def is_valid(self,c):
        for u,v in self.graph.edges():
            if c[u]==c[v]: return False
        return True

    def brute_force(self):
        nodes=list(self.graph.nodes())
        n=len(nodes)
        for k in range(1,n+1):
            for assign in self.generate_assignments(n, k):
                c={nodes[i]:assign[i] for i in range(n)}
                if self.is_valid(c): return k,c
        return n,{nodes[i]:i for i in range(n)}

    def color_graph(self):
        if not self.graph.nodes:
            messagebox.showerror("Помилка","Спершу створіть граф")
            return
        k,c = self.brute_force()
        messagebox.showinfo("Результат",f"Кількість кольорів: {k}")
        cmap = plt.get_cmap("hsv")
        colors = [cmap(i / k) for i in range(k)]
        node_colors = [colors[c[node]] for node in self.graph.nodes()]
        self.draw_graph(node_colors=node_colors)

    def draw_graph(self,node_colors=None):
        self.ax.clear()
        pos=nx.spring_layout(self.graph,seed=42)
        nx.draw(self.graph,pos,ax=self.ax,with_labels=True,node_color=node_colors or
"lightblue",node_size=600,edge_color="gray")
        self.canvas.draw()

if __name__ == "__main__":
    root=tk.Tk()
    GraphApp(root)
    root.mainloop()
```

Вміст файлу varCheck.py:

```
G = 42
N= 18
def get_variant_number():
    return (int(str(G) + str(N))%6)+1
def print_variant_number():
    print("Номер залікової книжки:", int(str(G) + str(N)))
    print("Мій варіант:",get_variant_number())
if __name__ == "__main__":
    print_variant_number()
```

Результат виконання програми:

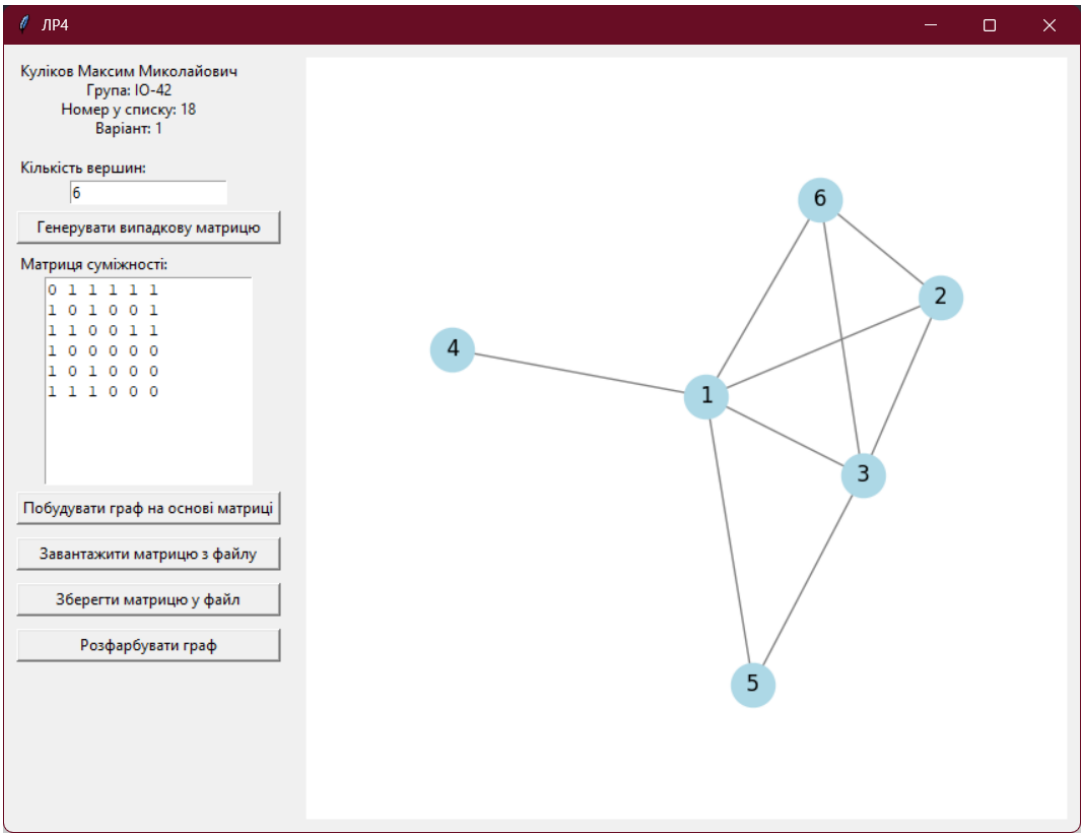


Рисунок 5.1 – результат виконання програми з утвореним графом

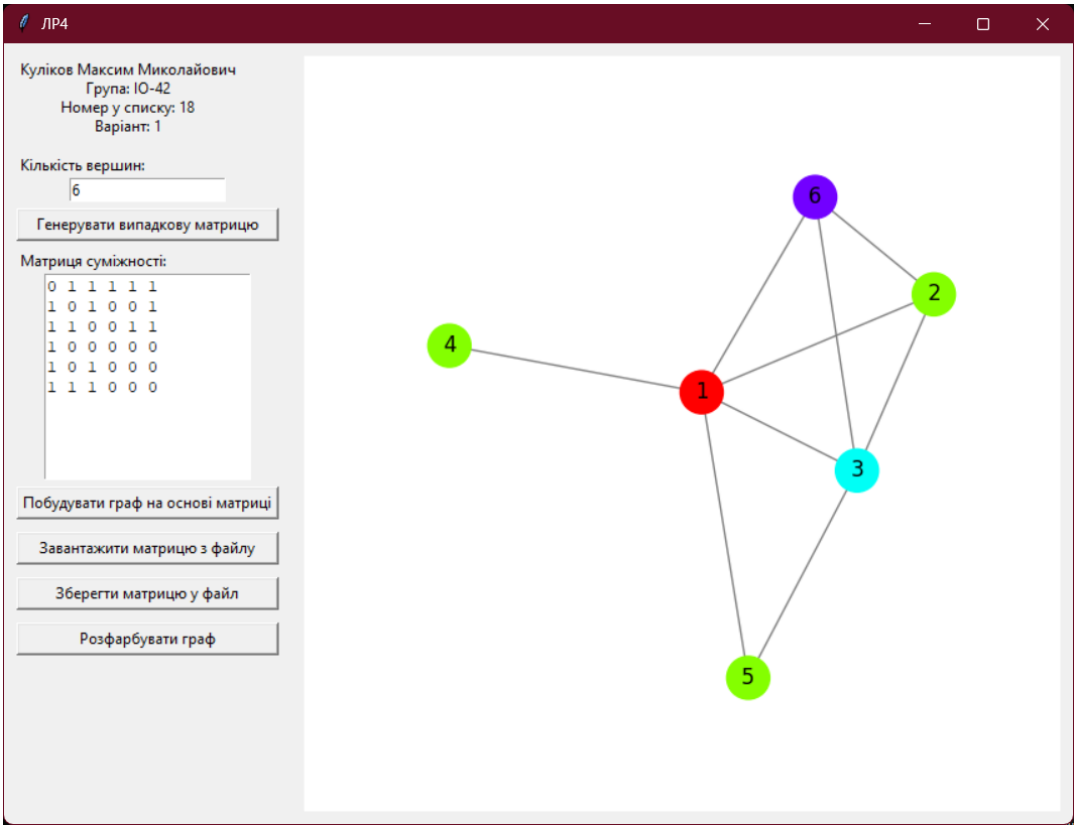


Рисунок 5.2 – результат виконання програми з утвореним розфарбованим

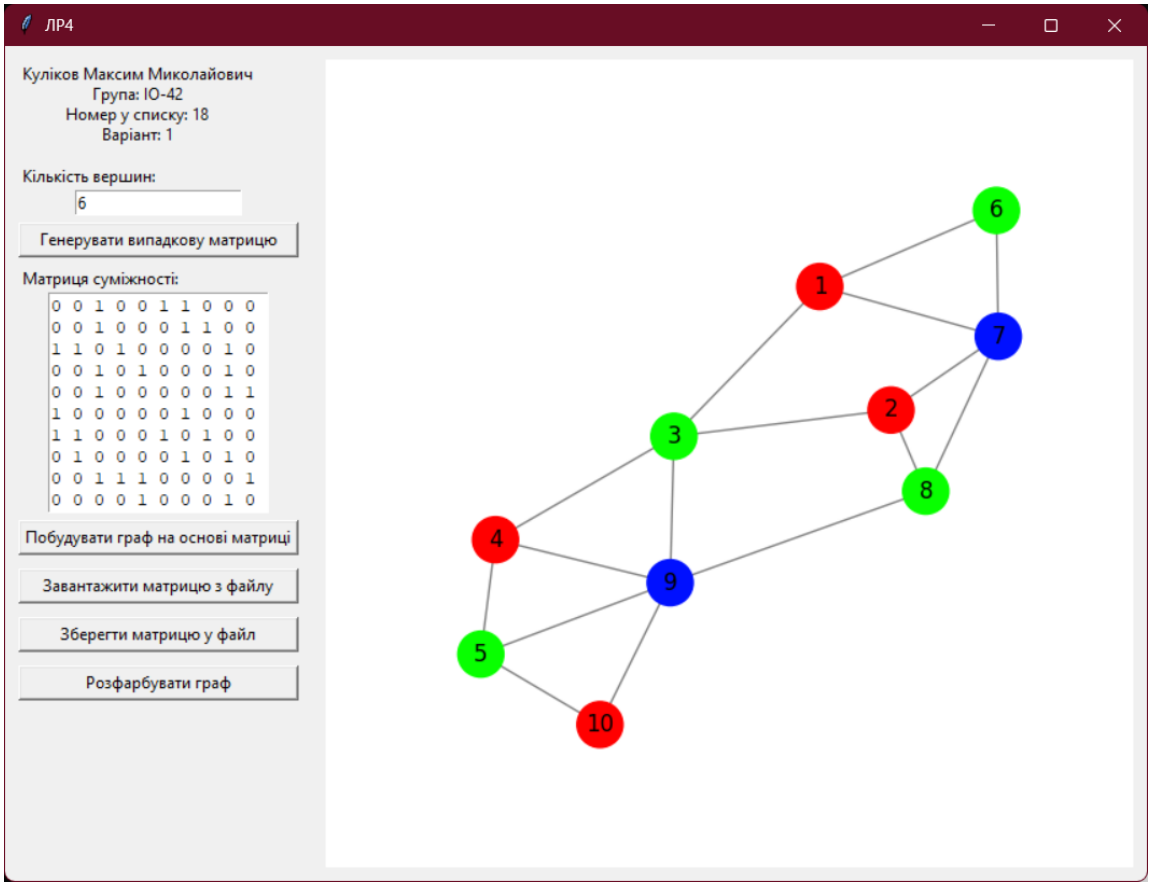


Рисунок 5.3 – результат виконання програми з розфарбованим графом за завданням

Висновки:

Отже, під час виконання лабораторної роботи я створив програму, яка дозволяє правильно розфарбовувати графи за допомогою методу прямого перебору, що дозволило мені краще засвоїти матеріал. Під час виконання проблем не виникало.